

مواصفة — إدارة علاقات العملاء (CRM) (وحدة تحكم العملاء للمشرفين)

القسم operations-it-06 النوع spec الحالة active المالك Founder آخر تحديث 21-06-2026

الحالة: مسودة · المالك: المنتج · آخر تحديث: 05-2026

وحدة تحكم داخلية مخصصة للموظفين فقط في Fly GACA لعرض وإدارة الأشخاص والمنظمات على المنصة — مستخدم الطيران واشتراكاتهم وخطوط الدعم ومسار الصفقات بين الشركات (B2B) لمدارس الطيران — في مكان واحد، بدلاً من التنقل بين وحدة تحكم Firebase ولوحة Stripe وصندوق البريد.

داخلية وتشغيلية. هذه الوحدة لموظفي فلاي قাকা فقط. تقرأ بيانات العملاء التي تحتفظ بها المنصة أصلاً (الحسابات، خطة entitlement ، حالة الفوترة) بموجب إشعار الخصوصية لدينا والتزاماتنا بنظام حماية البيانات الشخصية (PDPL). وهي ليست جزءاً من الموقع المنشور، ومضبوطة على noindex ، وتقع خلف الادعاء (claim) المخصص admin الموجود حصراً للموظفين (نفسه الذي يفرضه functions/school.js ؛ انظر 5s). وهي لا تكشف أبداً بيانات مستخدم آخر لمستدعي من خارج الموظفين.

1. السبب

اليوم، تتطلب الإجابة عن "من هذا العميل، وعلى أي خطة هو، وماذا طلب متأ؟" المقابلة اليدوية بين ثلاث أدوات:

- وحدة تحكم Firebase للحساب (users/{uid})، الذي يحمل حقل خطة entitlement).
- لوحة Stripe (RevenueCat للأجهزة المحمولة) للاشتراك.
- البريد الإلكتروني (i@flygaca.com) لكل ما كتبه العميل.

ومع تفعيل المرحلة الخامسة (المال ومدارس الطيران) للخطط المدفوعة وجلبها منظمات تشتري المقاعد، يزداد هذا سوءاً بسرعة. نحتاج إلى سطح واحد قراءته في الغالب يربط بين الحساب ← الاستحقاق ← الفوترة ← الدعم ← مسار الصفقات، بحيث يصبح الدعم أسرع، ومعدل التسرب مرتباً، ولا تضيق خطوط الـ B2B بين الشقوق. وحدة الـ CRM هي رؤية المشغل للعميل؛ تعيد استخدام البيانات التي يولدها بقية المنتج أصلاً بدلاً من اختراع مصدر حقيقة جديد.

2. المستخدمون

- الموظفون (admin) — فريق فلاي قাকা. الادعاء المخصص admin موجود أصلاً: يفرضه functions/school.js عبر assertAdmin(req) (req.auth.token.admin === true)، ويُضبط بـ admin.auth().setCustomUserClaims(uid, { admin: true }) . تعيد حوال الـ CRM القابلة للاستدعاء استخدام النمط نفسه تماماً. وتجري الكتابات ذات الصلاحيات أصلاً من جانب الخادم عبر Admin SDK

(`grantSchoolLicence` ، وخطافات Stripe/RevenueCat)، وتمنع `firestore.rules` جميع القراءات العابرة بين المستخدمين. وهم المستخدمون الأساسيون — وفي الإصدار الأول، الوحيدون — لهذه الوحدة.

- المبيعات/العمليات (دور مستقبلي) — ادعاء `crm` أضيق نطاقًا لمن يعمل على مسار الصفقات لكن لا ينبغي أن يرى مثلًا معرّفات الفوترة الخام. خارج نطاق الإصدار الأول؛ ويترك نموذج البيانات مجالًا له.
- العميل — موضوع السجل، لا مستخدم للوحة. فهو إمّا طيار (`users/{uid}` موجود) أو، لاحقًا، منظمة مدرسة.

3. النطاق

ضمن النطاق (الإصدار الأول)

- قائمة عملاء: كل حساب طيار مع الخطة وحالتها ولمحة سريعة عن آخر نشاط وتاريخ التسجيل؛ قابلة للبحث بالبريد الإلكتروني/الاسم؛ وقابلة للتصفية بالخطة والحالة (تجريبي / نشط / متأخر السداد / ملغى / مجاني).
- عرض تفاصيل العميل يربط، للقراءة فقط:
 - أساسيات الحساب من `users/{uid}` (الاسم المعروض، البريد الإلكتروني، اللغة المحلية، تاريخ الإنشاء).
 - الخطة من حقل `users/{uid}.entitlement` (الخطة، الحالة، المدة/الانتهاء).
 - ملخص الفوترة (عميل/اشتراك Stripe، آخر فاتورة) — يُقرأ عبر دالة سحابية، وليس مفاتيح Stripe من جانب العميل.
 - ملاحظات دعم العميل (أدناه).
- ملاحظات داخلية: يمكن للموظفين إلحاق ملاحظات مؤرّخة بعميل (الشيء الوحيد الذي *تكتبه* وحدة الـ CRM).
- مسار صفقات B2B (الخيوط): لوحة كانبان خفيفة لمدارس الطيران المحتملة — المرحلة، جهة الاتصال، تقدير القيمة، الإجراء التالي — تغذّي محرك "Fly GACA for Schools" في المرحلة الخامسة.
- نظرة عامة: بضعة عدّادات تشغيلية (الاشتراكات النشطة، التجارب المنتهية خلال 7 أيام، المتأخّرة السداد، الخيوط المفتوحة).

خارج النطاق (الإصدار الأول)

- تعديل الفوترة. تبقى المبالغ المستدّدة وتغييرات الخطط والإلغاءات في Stripe / بوابة العميل الخاصة. توقّر وحدة الـ CRM روابط خارجية؛ ولا تعدّل الفوترة.
- تعديل حساب العميل أو استحقاقه بما يتجاوز الملاحظات. يُكتب حقل `entitlement` فقط عبر خطافات الفوترة (Stripe / RevenueCat) و `grantSchoolLicence` — يجب ألا تصبح وحدة الـ CRM كاتبًا ثانيًا ينحرف عن الفوترة.
- البريد بالجملة / أتمتة التسويق. يبقى البريد المعاملاتي في تدفق الفوترة؛ أمّا الحملات فقرار منفصل لاحق (المرحلة الثامنة).
- تكامل CRM خارجي (HubSpot/Salesforce). تُعاد دراسته فقط إذا تجاوز مسار الصفقات لوحة الكانبان الداخلية.
- إدارة المنظمة الذاتية لمالكي المدارس — فذلك سطح منتج المرحلة السادسة، وليس هذه الوحدة الداخلية.

4. نموذج البيانات (Firestore)

تبقى حقيقة العميل حيث تقيم أصلًا — فالحساب وخطته كلاهما على `users/{uid}` (حقول الملف الشخصي + حقل `entitlement` المكتوب من جانب الخادم). لا تضيف وحدة الـ CRM سوى مجموعات يملكها المشغل من جانب الخادم فقط.

```
crmNotes/{noteId}
  subjectType: 'user' | 'lead'
  subjectId: string           // uid أو leadId
  authorUid: string          // الموظف الذي كتبها
  body: string
  createdAt: ts

leads/{leadId}               // مدارس الطيران / المنظمات - B2B مسار صفقات
  orgName: string
  contactName: string
  contactEmail: string
  phone: string
  stage: 'new' | 'contacted' | 'demo' | 'proposal' | 'won' | 'lost'
  seatsEstimate: number
  valueEstimateSar: number   // العملة بالريال السعودي، الهلات الصحيحة اختيارية
  ownerUid: string          // مالك الصفقة من الموظفين
  nextAction: string
  nextActionAt: ts
  source: string             // 'inbound' | 'referral' | 'event' | ...
  createdAt, updatedAt: ts

# CRM: مشتقة/للقرائة فقط، لا تخزنها وحدة الـ
# users/{uid}                - ملف الحساب (يملكه المستخدم)
# users/{uid}.entitlement     - الخطة/الحالة/المدة (تملكها خطوات الفوترة)
# Stripe / RevenueCat sub    - تجلب حيا عبر دالة سحابية
```

لا تغييرات على المجموعات الموجهة للعملاء. `crmNotes` و `leads` من جانب الخادم فقط (تكتب عبر Admin SDK من دوال الـ CRM القابلة للاستدعاء)، مثل عدّادات `adeLQuota` الموجودة. ويمثّل مستند ملخص `crmIndex` (صفوف قائمة غير معيارية) تحسينًا اختياريًا لقائمة العملاء، ويؤجّل حتى تصبح القائمة بطيئة.

5. نموذج الأمان

- أعد استخدام الادعاء الموجود `admin`. لا حاجة إلى بنية مصادقة جديدة — فـ `functions/school.js` يبوب أصلًا `req.auth.token.admin` `assertAdmin(req)` `revokeSchoolLicence` / `grantSchoolLicence` `=== true`). وتستدعي كل دالة CRM قابلة للاستدعاء الفحص نفسه أولًا. ويُستحسن استخراج `assertAdmin` إلى مساعد مشترك (مثل `functions/auth.js`) بحيث يتشارك `school.js` و `crm.js` تعريفًا واحدًا.
- تمرّ كل قراءة وكتابة في الـ CRM عبر دالة سحابية محميةً بصلاحيّة المشرف، تستخدم Admin SDK ومن ثمّ تتجاوز `firestore.rules`. ولا تقرأ الوحدة أبدًا مستند `users/{uid}` لمستخدم آخر مباشرةً من العميل.
- تبقى `firestore.rules` كما هي — عزل صارم لكل مستخدم مع التقاط شامل بالرفض `match` `/{document=**}` وتقع `crmNotes` و `leads` ضمن ذلك الالتقاط الشامل ومن ثمّ تكون غير قابلة للقراءة/

- الكتابة من أي عميل، ولا يمكن بلوغها إلا عبر دوال المشرف القابلة للاستدعاء. لا حاجة إلى قاعدة جديدة موجهة للعميل (ولا ينبغي إضافة أي منها — فهذه بيانات مشغّل، لا بيانات مستخدم).
- تُجلب معرّفات الفوترة (معرّفات عميل/اشتراك Stripe، بيانات البطاقة الوصفية) من جانب الخادم وتُعاد كملّص أدنى؛ ولا تصل الأسرار الخام وبيانات الدفع الكاملة إلى المتصفّح أبدًا.
- قابلية التدقيق: تسجّل الملاحظات `authorVid` + `createdAt`؛ وتختتم تغييرات مرحلة الخيط `updatedAt` و `ownerVid`. هذا سطح ذو صلة بنظام حماية البيانات الشخصية (PDPL) — وينبغي أن يكون وصول الموظفين إلى البيانات الشخصية قابلًا للعزو.
- الوحدة مضبوطة على `noindex, nofollow` ومحمية بتسجيل الدخول، مثل هيكل `admin.html` الموجود.

6. الأسطح

- `admin.html` — ينمو من الهيكل الحالي إلى غلاف الوحدة، أو يُضاف `crm.html` جديد بجواره. (يُحسم في المرحلة الأولى؛ أعد استخدام مُختم الترويسة/التذييل المشترك في الحالتين — ولا تحرّر أبدًا عناصر الإطار يدويًا.)
 - نظرة عامة — عدّادات تشغيلية.
 - العملاء — القائمة + التفاصيل (الحساب · الاستحقاق · الفوترة · الملاحظات).
 - مسار الصفقات — لوحة كانبان للخيوط.
- `functions/crm.js` (جديد) — استدعاءات RPC قابلة للاستدعاء بصلاحيّة المشرف فقط:
 - `crmListCustomers({ query, plan, status, cursor })`
 - `crmGetCustomer({ uid })` — يربط الحساب + `entitlement` + ملّص الفوترة
 - `crmAddNote({ subjectType, subjectId, body })`
 - `crmSetLeadStage({ leadId, stage })` / `crmUpsertLead(lead)` / `crmListLeads()`
 - مُصدّرة من `functions/index.js` بجوار `createCheckoutSession` و `grantSchoolLicence` وبقية الدوال القابلة للاستدعاء.
- يعيد استخدام `functions/entitlements.js` ووحدات الفوترة الموجودة (`functions/stripe.js`، `functions/revenuecatWebhook.js`) لملّص الخطة/الفوترة، بدلًا من تكرار ذلك المنطق.

7. الأسئلة المفتوحة

- قائمة العملاء على نطاق واسع. لا يملك Firestore بحثًا نصّيًا كاملًا ولا ربطًا من جانب الخادم. ابدأ بالاستعلام عن `users` على حقل خطة `entitlement` (المجموعة المدفوعة هي الأصغر والأكثر إثارة للاهتمام) والإثراء لكل صف؛ وأعد دراسة `crmIndex` أو بحث خارجي إذا أصبح بطيئًا.
- `crm.html` منفصل أم توسيع `admin.html`؟ الميل نحو وحدة مشرف واحدة بعلامات تبويب لتجنّب غلاف مصادقة ثانٍ.
- ادعاء `crm` (مبيعات) أضيّق متميّز عن `admin` — لازم عندما يعمل غير مهندس على مسار الصفقات. يُؤجّل حتى يوجد مشغّل ثانٍ.

- تسجيل الوصول بموجب PDPL. هل نحتاج إلى تسجيل *قراءات* البيانات الشخصية، لا الكتابات فحسب؟ غالبًا نعم للوصول المدفوع/الموظفين — تأكد مع نطاق LAWYER-BRIEF.
- الخيوط ↔ الحسابات. عند تحويل خيط مدرسة (`grantSchoolLicence`)، كيف نربط `leads/{leadId}` بالمدرسة/الطلاب الناتجين؟ أضف `linkedSchoolId` بمجرد إنزال نموذج بيانات المدارس من `spec-instructor-dashboard.md`.

8. المراحل

1. غلاف الوحدة + المصادقة. أنشئ غلاف `crm.html / admin.html` المبوب بصلاحيات المشرف خلف الادعاء الموجود `admin` (مع إعادة استخدام / استخراج `assertAdmin`). (دون تغيير `firestore.rules` — تبقى `leads / crmNotes` تحت الالتقاط الشامل بالرفض، ولا تُبلغ إلا عبر الدوال القابلة للاستدعاء).
2. العملاء (للقراءة فقط). القائمة + التفاصيل تربط الحساب والاستحقاق و ملخص فوتر Stripe.
3. الملاحظات. ملاحظات داخلية للإلحاق فقط على عميل.
4. مسار الصفقات. لوحة كانبان للخيوط (المراحل، المالك، الإجراء التالي) — منحدر دخول B2B لـ "Fly GACA for Schools".
5. نظرة عامة. عدادات تشغيلية (نشط / تجريبي / متأخر السداد / خيوط مفتوحة).